MEDIUM-4：计算机系统内存与执行机制

阶段一：硬件拾荒者 —— 存储的物理层级

【任务 1.1：识破奸商】

1.

- **CPU**
  - **是什么**：CPU是计算机的运算核心和控制核心，是一块超大规模的集成电路，相当于计算机的 “大脑”
  - **提供什么**：CPU 提供了计算机运行所需的基础逻辑运算和任务调度能力
  - **有什么功能**：处理计算机软件数据，将人类的命令转化为机器语言，并对其他计算机设备（如内存、显卡、主板等）下达命令，从结构上看，主要由三部分组成：
    - 运算器：负责执行具体的计算任务
    - 控制器：负责分析指令，并根据需要给运算器等部件下达不同的命令
    - 寄存器与缓存：协调控制器和运算器之间的工作，负责暂存和处理大量数据，以提高处理效率
- **GPU**
  - **是什么**：GPU是一种专门用于高速数学计算的电子设备
  - **提供什么**：提供了极其强大的并行计算能力（与 CPU 倾向于使用较少的核心并以线性顺序执行复杂任务不同，GPU 拥有数百甚至数千个核心，能够同时对多个数据值执行相同的操作）
  - **有什么功能**：GPU 擅长处理计算密集型任务
    - 游戏与专业可视化：为复杂的3D游戏场景、视频编辑和医学影像等提供逼真的图形渲染
    - 人工智能与机器学习：AI 模型的训练和推理本质上是海量的矩阵运算，GPU 的并行处理能力使其成为 AI 时代的核心算力底座
- **主板**
  - **是什么**：主板是计算机最基本也最重要的部件之一，是一块布满线路和插槽的大型印刷电路板
  - **提供什么**：主板本身不直接进行复杂计算，但它提供了整个计算机系统的连接平台、电力分配中心和数据传输枢纽
  - **有什么功能**：主板的核心功能是连接并协调所有硬件的协同工作：
    - 硬件连接：CPU、显卡、内存、硬盘、键盘鼠标等所有硬件都必须连接在主板上才能正常工作
    - 信号与电力传输：从按下开机键开始，所有的信号传递、电力输送都要通过主板完成，主板上的供电电路会将电源的电力精准地分配给 CPU、内存等对电压要求极高的硬件
    - 数据通信：主板上的 芯片组 和各类 总线 构成了数据传输的“高速公路”，负责管理 CPU 与各种外设、存储设备之间的数据交换
    - 系统启动：主板上的 BIOS/UEFI 固件是计算机启动时加载的第一个程序，负责进行硬件自检并引导操作系统启动

2.

（1）手机上的512G内存指的是手机的存储空间（ROM/闪存），而电脑上的16G内存指的是运行内存（RAM）

（2）内存和主存基本上等价，全称叫做“随机存取存储器”（RAM），特点是读写速度极快，但断电后数据会丢失，它主要用于临时存放正在运行的程序和数据

（3）存储通常指外存（即辅助存储器），如固态硬盘，机械硬盘或手机闪存，特点是读写速度相对较慢，但断电后数据不会丢失

（关于内存和存储，我在后记也有提到）

3.

电源需要注意哪些：

（1）电源是把220V交流电转换为可以供CPU、显卡、硬盘等硬件可以用的稳定低压电流的一种硬件，同时还承担着过滤杂波、稳压和保护硬件的任务

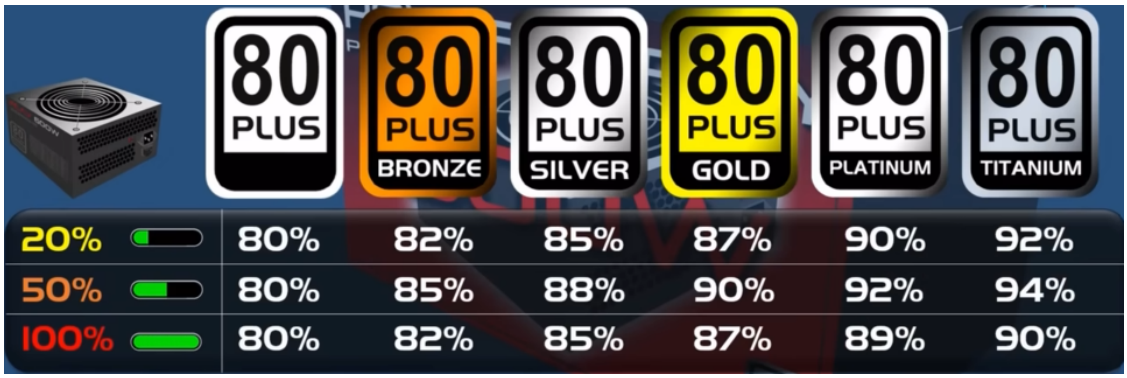
电源按大小分为SFX电源和ATX电源，SFX电源常给ITX主机(一种小型主机)使用，下面只讨论ATX电源：

（2）电源按线材分为：

- 直出线（非模组）电源：所有电线全部焊死在电源上
  - 优点：结构简单，价格便宜，不易损坏
  - 缺点：不用的线也会塞在机箱里面，整体冗余
- 半模组电源：主板、CPU供电线固定拔不掉，显卡、硬盘、大4pin风扇等线可以拔掉
  - 优点：性价比高
- 全模组电源：所有的电线都能自由插拔
  - 优点：机箱最干净整洁，散热性好

（3）80PLUS认证：

- 认证要求：在轻载（20%）、中载（50%）、满载（100%）三种情况下转换效率都 >= 80%，并且功率要达标



- 有认证的好处：
  - 省电
  - 发热少，噪声小
  - 更耐用，电源内部原件老化慢，寿命更长，也更稳定

不过需要注意的是：80PLUS认证只关注转换效率，不考虑电源的稳定性、纹波控制、质保年限等关键指标

4.

（1）i7 不一定比 i5 强，虽然在同代、同后缀的处理器中，i7 确实比 i5 强，但跨代或跨系列比较时，情况就完全不同了

- 代际差异大于级别：较新的 i5 在单核性能和综合表现上，完全可能超越前几代的老款 i7
- 后缀代表不同的功耗与定位： 标压高性能的 i5（如 HX 系列）性能会强于主打低功耗的 i7（如 U 或 P 系列）（Intel和AMD的后缀命名规则不同，以Intel为例，笔记本：HX>H>P>U>V；台式机：K、KF、F、KS、T、BFC（排名不分先后））

（2）40系显卡相比30系有显著的代际跃升，例如：40系原生支持 DLSS 3.0 帧生成技术，而 30系即便更新驱动也无法使用。在支持的游戏中，40系显卡能带来成倍的帧率提升，且画面更流畅，并且，得益于更先进的工艺，40系在性能大幅提升的同时，功耗控制得更好

（3）处理器和显卡的性能主要从以下三个维度入手：

- 看核心参数：
  - CPU：重点关注代数、核心/线程数（决定了多任务处理能力）、基础与睿频频率、以及三级缓存（L3）大小（直接影响游戏帧率和响应速度）
  - 显卡：重点关注架构代际、流处理器（CUDA核心）数量、显存容量与类型（如 GDDR6X 优于 GDDR6）、以及显存位宽（显存带宽由容量、类型和位宽共同决定，对高分辨率游戏至关重要）
- 看权威跑分：理论上跑分高的一般较好一些，但也不绝对，毕竟跑分测试是短暂的，并且只代表理论峰值性能，和实际体验多少存在差距
- 看实际场景表现：
  - 游戏：不仅看平均帧率，更要关注 1%LOW帧率（最低帧）和帧生成时间
  - 创作：应关注软件的实际导出时间或生成速度，40系显卡在这些 AI 负载下的吞吐量通常是30系的近两倍

5.

（1）”其上搭载比 i5处理器 还多i2的 i7 级CPU“：啥叫 “多 i2 ” ？ 况且，泥也妹没说CPU的代际和系列啊

（2）”16G的高端内存“：16G内存也叫高端?

（3）”性能比上一代高出一半的豪华GPU“：显卡跨越一代显然不可能有50%的提升，肯定是虚假宣传!

（4）”我再送你台 512G 内存的手机“：512G应该是存储，真要是内存，你不早干垮黄仁勋了

（5）”外加一块 1.25T 的超大硬盘“：？ 有1.25T这个规格吗...（常见的标准容量1T、2T）

【任务 1.2：你问我答】

1.常见存储介质

- **SRAM(静态随机存取存储器)**
  - **应用部位**：主要用于CPU 内部的高速缓存，如 L1、L2、L3 缓存
  - **主要特点**：
    - 速度极快且无需刷新
    - 成本高，容量小：内部结构复杂，占用芯片面积大，造价昂贵
    - 易失性：断电后数据会丢失
  - **特点起到的作用**：因为 CPU 的运算速度极快，为了防止CPU 就会被迫等待，SRAM 被放置在离 CPU 核心最近的地方，存放 CPU 最频繁使用的指令和数据，从而极大减少 CPU 的等待时间，提升整体运算效率
- **DRAM(动态随机存取存储器)**

- **应用部位**：主内存（内存条），如电脑上的 DDR4/DDR5，手机上的 LPDDR
- **主要特点**：
  - 需要周期性刷新：结构极其简单（1个晶体管+1个电容），但因为电容会缓慢漏电，必须每隔几毫秒刷新一次来维持数据（所以叫 “动态“ 嘛）
  - 性价比高：相比 SRAM，它的集成度极高、成本低、容量大，但读写速度稍慢
  - 易失性：断电后数据会丢失（和SRAM相同）
- **特点起到的作用**：它是 CPU 和硬盘之间的“核心工作台”,当一个软件被打开时，数据会从硬盘加载到 DRAM 中，DRAM的大容量保证了多个程序同时运行的可行性，而相对较快的速度则保证了系统运行的流畅度

- **NAND Flash (NAND 闪存)**
  - **应用部位**：用于固态硬盘（SSD）、手机/平板的机身存储（UFS/eMMC）、U盘和 SD 卡
  - **主要特点**：
    - 非易失性：断电后数据能长期保存（通常可达十年以上）
    - 无机械部件：纯半导体芯片，抗震抗摔，零噪音，功耗低
    - 有擦写寿命：存储单元有写入次数限制（P/E周期），需要通过主控芯片的“磨损均衡”算法来延长寿命
  - **特点起到的作用**：可以用来永久保存操作系统、应用程序、照片和视频等，它的普及彻底取代了机械硬盘作为系统盘的地位，让电脑实现了秒级开机和软件的瞬间加载
- **HDD (机械硬盘)**
  - **应用部位**：常用于台式机、监控设备以及大型数据中心
  - **主要特点**：
    - 机械结构：依靠高速旋转的磁性盘片和移动的磁头来读写数据，因此存在物理寻道延迟，读写速度慢，且怕震动
    - 容量极大：单盘容量可达数十 TB，单位存储成本远低于 SSD
    - 非易失性：断电后数据永久保存
  - **特点起到的作用**：由于速度较慢，它不适合用来运行系统和大型游戏，但它凭借极低的成本和超大的容量，非常适合用来做海量数据的备份、归档以及存储高清电影等不常变动的文件

2.

- **寄存器**
  - **是什么**：寄存器是计算机中速度最快、但容量极小的存储单元，它主要用于暂时存放CPU正在处理的数据、指令或内存地址
  - **位于哪里**：完全位于CPU芯片内部，与运算器和控制器直接相连
- **高速缓存**
  - **是什么**：高速缓存是一种存取速度极快、但容量较小的存储器，为了解决CPU处理速度远快于内存读写速度的“不匹配”问题，它会自动把CPU近期最可能用到的数据提前存起来，让CPU不用每次都去慢速的内存里找数据，从而大幅提升电脑运行效率
  - **位于哪里**：主要集成在CPU芯片内部（如L1，L2缓存），或者封装在CPU内部（如L3缓存），紧挨着CPU核心，距离极近
- **只读存储器（ROM）**
  - **是什么**：是一种断电后数据不会丢失的存储器，特点是 “只能读出，不能随意写入”，通常用来固化存储计算机的基本输入/输出系统（BIOS）等核心固件
  - **位于哪里**：它通常被做成一个小巧的集成电路芯片，直接焊接或插在主板上

3.

关于DRAM：

DRAM的物理结构：DRAM 的核心存储单元结构非常简洁，通常被称为 1T1C 结构，即由 1个晶体管（Transistor）和 1个电容（Capacitor）组成

- *晶体管*：负责控制访问。它连接在电容和外部电路之间，只有当晶体管被激活（导通）时，外部电路才能对电容进行读写操作
- *电容*：负责存储数据。通过内部存储电荷的有无来代替二进制数据，其中有电荷代表 1，无电荷代表 0

在芯片内部，海量的 1T1C 存储单元会排列成一个二维矩阵（存储阵列）。同一行单元的晶体管栅极连接到一条 *字线（Word Line）*，用于控制该行的开关；同一列单元则连接到一条 *位线（Bit Line）*，用于传输数据

DRAM的基本工作原理：DRAM的工作主要围绕数据的写入、读取和刷新三个基本操作展开：

- *写入*：当需要写入数据时，系统会激活目标单元所在的字线，使晶体管导通。随后，通过位线施加相应的电压：写入“1”时，向电容充电；写入“0”时，将电容上的电荷释放（放电）
- *读取*：读取时，同样先激活字线导通晶体管。此时，电容会与位线共享电荷，导致位线产生一个微小的电压变化。这个变化会被一个称为 *感测放大器（Sense Amplifier）* 的电路检测并放大，从而判断出存储的是“0”还是“1”
- *刷新*：这是 DRAM 最核心的特性。由于现实中的晶体管并非完美绝缘体，存在漏电电流，即使晶体管关闭，电容上存储的电荷也会随时间自然流失。如果不加干预，数据将在几毫秒内丢失。
  为了防止数据丢失，DRAM 控制器必须 周期性地（通常每 64 毫秒）对所有存储单元执行一次刷新操作。刷新的本质就是：读取数据并立即回写，在不向外部输出数据的情况下，为电容重新充电，恢复其满电荷状态

4.

Cache（高速缓存）的根本原理是基于**程序局部性原理**

程序局部性原理具体包含以下两个核心维度：

- **时间局部性**：指一个数据或指令一旦被访问，那么在不久的将来它很可能被再次访问，例如，程序中的循环变量、被反复调用的子程序等，都表现出明显的时间局部性
- **空间局部性**：指当程序访问了某个存储单元后，在不久的将来，它很可能会访问与该单元地址相邻的存储单元。例如，程序在顺序执行指令或遍历数组时，就会表现出空间局部性

基于上述原理，当CPU需要读取数据时，会首先去Cache中查找：

- 命中（Hit）：如果数据存在于Cache中，CPU会立即以极高的速度读取并处理，无需访问主存
- 未命中（Miss）： 如果数据不在Cache中，CPU只能以较慢的速度从主存中读取。同时，Cache会将该数据及其相邻的数据块（Cache Line）一并调入缓存中，以备后续的访问

通过这种机制，现代CPU读取数据的Cache命中率通常可以达到90%左右，使得整个存储系统既具备了Cache的高速度，又拥有了主存的大容量

5.

在检测一个N * N的矩阵是否为正确的密钥矩阵时，由c[j] [k] = a [j] [l] * b[l] [k]可知，可以用多层遍历的方法：

```
for(int j = 0;j<N;j++){
    for(int k = 0;k<N;k++){
        for(int l = 0;l<N;l++){
            c[j][k] += a[j][l]*b[l][k];
        }
    }
}
```

然而，矩阵的存储方式是按行存储，而B矩阵却是按列遍历，也即每取一个b，就要跳过（N - 1）个地址，如果(N - 1)大于了Cache的大小，CPU就会不得不到内存中读取，从而降低了效率

在保证结果不变的前提下，我们可以尝试这样优化：

```
for(int j = 0;j<N;j++){
    for(int l = 0;l<N;l++){
        for(int k = 0;k<N;k++){
            c[j][k] += a[j][l]*b[l][k];
        }
    }
}
```

不难看出：这利用了**空间局部性原理**，在调换了遍历次序后，正在被读取的b和与之相邻的地址都被存储在Cache中，使得CPU的取用更加方便，大大提高了效率

能不能用**时间局部性原理**来优化呢?

AI给出了答案：

当 N 很大时，B 的一整行都塞不进 Cache，既然整个 B 装不下，那就切成小块，让每个小块装得下

外层三循环负责轮流把每个小块搬进 Cache，内层三循环负责在小块内部做实际的乘法

```
for (ii = 0; ii < N; ii += BSIZE)
  for (jj = 0; jj < N; jj += BSIZE)
    for (kk = 0; kk < N; kk += BSIZE)
      for (i = ii; i < min(ii+BSIZE, N); i++)
        for (k = kk; k < min(kk+BSIZE, N); k++) {
          double r = A[i][k];
          for (j = jj; j < min(jj+BSIZE, N); j++)
            C[i][j] += r * B[k][j];
        }
```

阶段二：虚拟空间潜行 —— MMU 与页表机制

【任务 2.1：揭开虚拟的面纱】

1.

物理地址：物理地址是实际存在于物理内存（RAM）硬件单元上的真实唯一编号，它是计算机硬件（如CPU和内存条）真正用来定位和访问数据的地址

虚拟地址：虚拟地址是程序在运行时所使用的逻辑地址，而不是计算机内存条上真实的地址

映射关系：多个虚拟地址（甚至不同进程的虚拟地址）可以映射到同一个物理地址，然而并不是所有的虚拟地址在任何时刻都对应着物理内存，并且连续的虚拟地址，在物理内存中往往是不连续的（多对一/零，非连续映射）

操作系统引入 “虚拟内存” 的优点：

- 保证安全性：每个运行的进程都有自己独立的虚拟地址空间，这意味着一个进程中的代码无法随意访问或修改另一个进程或操作系统内核所使用的物理内存
- 提供连续的地址空间：操作系统将多个非连续的物理内存块映射为一段连续的虚拟地址，极大地简化了内存管理的复杂性
- 提升内存利用率：多个进程还可以共享同一份物理内存中的代码或共享库，避免了重复加载，从而最大化地利用系统资源
- 扩大可用空间：当物理内存不足时，操作系统会将不常用的数据暂时转移到硬盘上，从而腾出物理内存来处理更重要的任务
- 简化软件开发

2.

（1）

每个进程的虚拟地址空间的特性：

- 独立性：每个进程的虚拟地址空间都是独立且完整的
- 存在严格的功能分区：每个进程的虚拟地址空间都分为 内核空间 和 用户空间，它们又各自分为许多功能不同且大小固定的区块
- 连续性：每个进程的虚拟地址空间都是一个从低地址（0x0000 0000）到高地址（0xFFFF FFFF）的连续范围

内核空间：1G；用户空间：3G；总共：4G

（2）

虚拟地址空间的基本组成单位是 **页**，其标准大小通常为 4KB

（3）

虚拟地址空间中的页可以被划分到：代码段、数据段、BSS段、堆、栈、文件映射以及匿名映射区

（4）

页

- **是什么**：页是组成**虚拟地址空间**的基本单位，是操作系统将进程的逻辑内存划分成的固定大小的块
- **作用**：
  - 按需加载，当进程需要更多内存时，操作系统以页为单位分配物理页框
  - 提高内存利用率，操作系统可以灵活地将进程中的若干页分别装入多个不相邻的物理页框中，这有效消除了外部内存碎片问题，使得内存分配更加高效和整齐
  - 提供进程隔离与内存保护，每个进程都拥有自己独立的虚拟地址空间和专属的页表，这种机制确保了不同进程之间的地址空间相互隔离，防止了进程越界访问其他进程或内核的内存，从而保障了系统的安全与稳定

页框

- **是什么**：页框是组成**物理内存**的基本单位，是RAM被划分成的与页大小相同的固定块
- **作用**：作为页的物理载体，当页被调入内存时，必须分配一个空闲的页框来存放它；操作系统通过维护空闲页框列表来管理物理内存的分配与回收；页框的大小必须与页的大小完全一致，以确保映射的正确性

共同点：

- 大小相同且固定
- 都是按需分配

【任务 2.2：认识页表管理】

1.

（1）

显然，页表和页不是一个东西，页表是操作系统用于实现虚拟内存管理的核心数据结构，它的根本作用是：记录并维护虚拟内存中的 “页” 与物理内存中的 “页框” 之间的映射关系

本质上，页表就是一个由 页表项 组成的数组或链表结构，包括：页、物理页框号、状态与控制标志位（存在位、访问权限位、脏位、访问位）

（2）

MMU（内存管理单元）：MMU是位于 CPU 内部的一块硬件单元，它是整个地址翻译过程的 *总指挥和执行者*

其核心职责为：

- 接受请求：截获CPU发出的虚拟地址
- 执行翻译：负责将虚拟地址映射为真实的物理地址
- 权限检查：在翻译的同时，检查该进程是否有权限访问该内存地址（只读、可写、可执行），防止非法访问
- 异常处理：若找不到对应的物理地址，MMU会触发 ”缺页异常“，交由操作系统处理

TLB（转译后备缓冲区）：TLB 是集成在 MMU 内部（或 CPU 内部）的一块极高速的硬件缓存，其核心职责是 *加速地址* 翻译：

- 缓存热点数据：TLB专门用来缓存最近、最常被访问的 虚拟页 到 物理页框 的映射关系（即页表项）
- 匹配速度极快：因为它的容量很小但速度极快（访问速度几乎与 CPU 一致），当 MMU 需要翻译地址时，会首先去 TLB 中查找，如果命中（TLB Hit），MMU 就能瞬间拿到物理地址，完全跳过查内存的过程

（3）

进程用来查找物理页框的单级或多级页表存储在**物理内存**中，通常而言，一般存放在系统空间的页表区

用来记录页表 “物理基地址” 的特定寄存器是**CR3**，也称为页目录基址寄存器

（4）

二级页表是一种分层虚拟内存管理机制，其核心思想是将庞大的单级页表进行分级拆分，通过 “一级页表”（页目录）和 “二级页表”（页表）两级结构来实现按需加载

对比单级页表，二级页表的优势主要体现在 ”节省内存空间“ 和 ”降低对连续物理内存的要求“

- 节省内存空间：二级页表将虚拟地址划分为一级页号、二级页号和页内偏移量。一级页表仅包含少量条目，每个条目指向一个二级页表。只有当进程真正使用到某一段虚拟地址时，操作系统才会为其分配对应的二级页表。对于未使用的虚拟地址空间，一级页表中的对应项为空，无需创建二级页表。这种“按需分配”的机制使得页表的实际内存占用大幅减少，有时能节省99%以上的空间
- 无需大块连续物理内存：在二级页表机制下，只有体积很小的一级页表需要常驻内存。而数量众多的二级页表可以离散地存放在物理内存的任何空闲页框中，一级页表中的条目只需记录对应二级页表的物理基地址即可，从而完美解决了大块连续内存分配难的问题

总的来说，二级页表采用了 “以时间换空间” 的策略，极大提高了内存的利用效率

（5）

一个32位的虚拟地址具体被划分为以下三个部分：

- **一级页号**：
  - 位数：10位，位于虚拟地址的最高位（即第22位至第31位）
  - 含义：作为一级页表的索引，硬件利用这10位去查找页目录表中的页目录项（PDE），从而获取对应二级页表的物理基地址
- **二级页号**：
  - 位数：10位，位于虚拟地址的中间部分（第12位至第21位）
  - 含义作为二级页表的索引，硬件结合上一步获取的二级页表基地址，利用这10位去查找具体的页表项（PTE），从而获取目标数据所在的物理页框号（物理页帧基地址）
- **页内偏移量**
  - 位数：12位，位于虚拟地址的最低位（即第0至第11位）
  - 含义：用于定位物理页框的具体字节位置，由于典型的页面大小为4KB（即 2^{12} 字节），这12位偏移量在虚拟地址和物理地址中是完全相同的，最终会直接与物理页框号拼接，生成最终的物理地址

2.

在补全函数的过程中，我出现了一系列卡顿

首先，因为对 page_dir_index 和 page_table_index 的理解不够深刻，我没能很快将其与数组下标访问语法联系起来[哭笑]，在 PDE page_directory[PAGE_DIR_SIZE]; 这句代码的提示之下，才幡然醒悟

其次，对于任务 “查询” 我一开始也有些疑惑，联想到这是一个写入函数，最终才确定是要求我检查前置条件

根据注释以及题目中其他结构体、宏定义的提示，我最终还是完成了该函数

代码本身的逻辑都不复杂，在此不过多赘述

对照题目给出的 sample output，还需要加上printf（题目中的printf(" [*] Checking access permissions...\n");位置好像有问题。。）

代码如下（只节选了需要补充的部分）：

```
void translate_address(uint32_t virtual_address, bool is_write_access) {//请你完成这个尚未完成的函数
    printf("----------------------------------------\n");
    printf("Translating Virtual Address: 0x%08X (%s access)\n", virtual_address, is_write_access ? "WRITE" : "READ");

    // --- 步骤1: 从虚拟地址中提取索引和偏移 ---
    uint32_t page_dir_index ;
    uint32_t page_table_index;
    uint32_t offset;
    // 利用位运算（移位 >> 和 掩码 &）提取对应部分
    // 请补全代码写在TO DO与END OF TO DO之间:
    // TO DO:

    page_dir_index = virtual_address >> (PAGE_TABLE_BITS+OFFSET_BITS);
    page_table_index = (virtual_address >> OFFSET_BITS) & (PAGE_TABLE_SIZE - 1);
    offset = virtual_address & (PAGE_SIZE - 1);

    // END OF TO DO:

```

```
    printf(" -> Page Dir Index: %u (0x%X)\n", page_dir_index, page_dir_index);
    printf(" -> Page Table Index: %u (0x%X)\n", page_table_index, page_table_index);
    printf(" -> Offset: %u (0x%X)\n", offset, offset);

    // --- 步骤2: 查询页目录表 ---
    // 请补全代码写在TO DO与END OF TO DO之间:
    // TO DO:

    PDE pde = page_directory[page_dir_index];

    printf("  [*] Checking Page Directory Entry %u...\n", page_dir_index);

    if(!pde.present_bit) {
       printf(" [!] FAULT: Page Directory Entry not present. (Segmentation Fault)\n");
        return;
    }

    printf("      -> PDE is present. Page table base address: 0x%p\n", (void*)pde.page_table_base);

    // END OF TO DO:

    // --- 步骤3: 查询页表 ---
    // 请补全代码写在TO DO与END OF TO DO之间:
    // TO DO:

    PTE pte = pde.page_table_base[page_table_index];

    printf("  [*] Checking Page Table Entry %u...\n", page_table_index);

    if (!pte.present_bit) {
        printf(" [!] FAULT: Page Table Entry not present. (Page Fault)\n");
        return;
    }
    printf("      -> PTE is present. Frame number: %u (0x%X)\n", pte.frame_number, pte.frame_number);

    // END OF TO DO:

    // --- 步骤4: 检查访问权限 ---
    // 请补全代码写在TO DO与END OF TO DO之间:
    // TO DO:

    printf("  [*] Checking access permissions...\n");

    if (is_write_access && !pte.rw_bit) {
        printf("  [!] FAULT: Write attempt on a read-only page. (Protection Fault)\n");
        return;
    }

    printf("      -> Access granted.\n");

    // END OF TO DO:

```

```
71
72      // --- 步骤5: 计算最终的物理地址 ---
73      // 请补全代码写在TO DO与END OF TO DO之间:
74      // TO DO:
75
76      uint32_t physical_address;
77      physical_address = (pte.frame_number << OFFSET_BITS) | offset;
78
79      // END OF TO DO:
80      printf("  [SUCCESS] Translation complete.\n");
81      printf("  Virtual Address 0x%08X  =>  Physical Address 0x%08X\n",
    virtual_address, physical_address);
82  }
```

运行结果如下图:

```
PS D:\c\.vscode>  ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\Mark\.vscode\extensions\ms-vscode.cpptools-1.34.4-win32-x64\debugAdapters
soft-MIEngine-In-yped2ud4.j4v' '--stdout=Microsoft-MIEngine-Out-q4zf1nh4.nr1' '--stderr=Microsoft-MIEngine-Error-xo4jhki5.jld' '--
bgExe=D:\c\mingw64\bin\gdb.exe' '--interpreter=mi'
    -> PDE is present. Page table base address: 0x0000000000A44B50
  [*] Checking Page Table Entry 1...
      -> PTE is present. Frame number: 26 (0x1A)
  [*] Checking access permissions...
      -> Access granted.
  [SUCCESS] Translation complete.
  Virtual Address 0x00001A2B  =>  Physical Address 0x0001AA2B
----------------------------------------
Translating Virtual Address: 0x00001A2B (WRITE access)
 -> Page Dir Index: 0 (0x0)
 -> Page Table Index: 1 (0x1)
 -> Offset: 2603 (0xA2B)
  [*] Checking Page Directory Entry 0...
      -> PDE is present. Page table base address: 0x0000000000A44B50
  [*] Checking Page Table Entry 1...
      -> PTE is present. Frame number: 26 (0x1A)
  [*] Checking access permissions...
      -> Access granted.
  [SUCCESS] Translation complete.
  Virtual Address 0x00001A2B  =>  Physical Address 0x0001AA2B
----------------------------------------
Translating Virtual Address: 0x00002048 (WRITE access)
 -> Page Dir Index: 0 (0x0)
 -> Page Table Index: 2 (0x2)
 -> Offset: 72 (0x48)
  [*] Checking Page Directory Entry 0...
      -> PDE is present. Page table base address: 0x0000000000A44B50
  [*] Checking Page Table Entry 2...
      -> PTE is present. Frame number: 143 (0x8F)
  [*] Checking access permissions...
  [!] FAULT: Write attempt on a read-only page. (Protection Fault)
----------------------------------------
Translating Virtual Address: 0x00003555 (READ access)
 -> Page Dir Index: 0 (0x0)
 -> Page Table Index: 3 (0x3)
 -> Offset: 1365 (0x555)
  [*] Checking Page Directory Entry 0...
      -> PDE is present. Page table base address: 0x0000000000A44B50
  [*] Checking Page Table Entry 3...
 [!] FAULT: Page Table Entry not present. (Page Fault)
----------------------------------------
Translating Virtual Address: 0x00804000 (READ access)
 -> Page Dir Index: 2 (0x2)
 -> Page Table Index: 4 (0x4)
 -> Offset: 0 (0x0)
  [*] Checking Page Directory Entry 2...
 [!] FAULT: Page Directory Entry not present. (Segmentation Fault)
```

【任务 2.3：深入理解内存与页表管理】

1.

与内存相关的寄存器有:

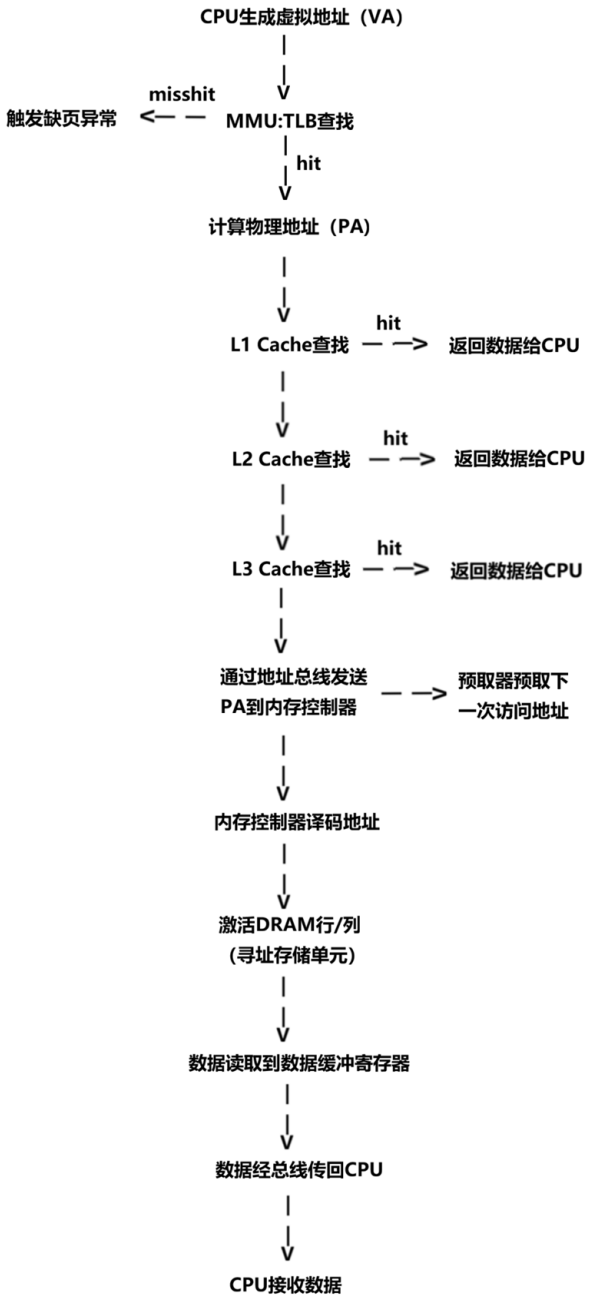
- **内存寻址与指针寄存器**：用于存放内存地址或参与地址计算
  - 基址寄存器：常用于存放存储器的基地址，作为内存指针使用
  - 变址寄存器：用于存放存储单元在段内的偏移地址。在数组遍历或字符串操作中，它们常与基址寄存器配合，实现“基址+变址”的灵活寻址

    - 堆栈指针寄存器：指向当前堆栈的栈顶位置。在函数调用、局部变量分配和中断响应时，用于自动维护堆栈的 压入 和 弹出 操作
    - 基址指针寄存器：通常指向当前堆栈帧的基地址，用于在函数执行期间稳定地访问局部变量和函数参数
- **内存分段管理寄存器**：负责划定逻辑内存空间
    - 代码段寄存器：存放当前执行代码的段基址
    - 数据段寄存器：存放全局变量和静态变量的数据段基址
    - 堆栈段寄存器：存放当前堆栈的段基址，与 SP 配合使用
    - 附加数据段寄存器：提供额外的数据段空间，常用于字符串操作或特定的数据结构访问
- **内存管理与地址转换寄存器**：实现虚拟内存的核心
    - 页目录基址寄存器：这是最关键的内存管理寄存器之一，它专门用来存放当前进程 页目录表 在物理内存中的起始基地址，当操作系统切换进程时，会修改 CR3 的值，从而实现不同进程间虚拟地址空间的隔离
    - TLB相关寄存器：TLB 是硬件中用于加速虚拟地址到物理地址转换的高速缓存。MMU 内部会使用隐藏的 TLB 标记/数据寄存器来存储最近使用的页表映射，避免每次都去内存中查询页表
- **内存访问与状态寄存器**
    - 控制寄存器：包含了控制 CPU 特性的标志位，如分页启用位、保护启用位等，决定了系统当前是运行在实模式、保护模式还是开启了虚拟内存分页机制
    - 状态/标志寄存器：虽然主要用于记录 ALU 运算结果，但在内存操作中，如果发生越界访问或权限违规（如写只读页），硬件会设置相应的异常标志，从而触发操作系统的缺页异常（Page Fault）或保护异常（Protection Fault）处理程序

2.

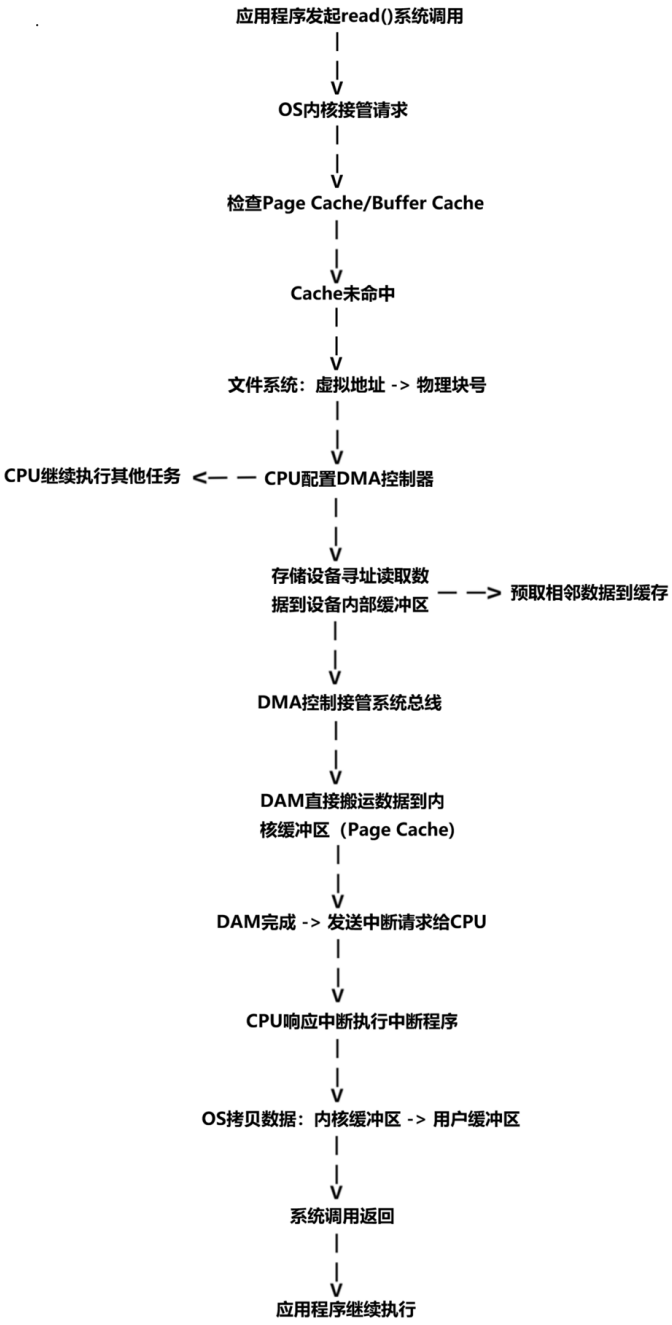
一次从内存中完整读取数据的过程：

- CPU首先生成虚拟地址（VA），通过MMU进行TLB查找（若未命中，则触发缺页异常），将虚拟地址转换为物理地址
- 物理地址生成后，依次查询L1、L2、L3，一旦某一级命中，数据直接返回给CPU，若未命中，则物理地址通过地址总线发送给内存控制器，此时，内存控制器会立即启动访问流程，并可能同时触发预取器预测下一次访问的地址，以提前准备数据，减少后续延迟
- 内存控制器首先对物理地址进行译码，找到Bank地址，行地址，列地址
- 在列地址被选中后，DRAM会从存储阵列中读取整行数据到 行缓冲区，再由内存控制器将 行缓冲区 中的数据按位宽分批通过数据总线传回CPU



一次从存储中完整读取数据的过程：

- 应用程序发起read()系统调用，然后CPU从用户状态切换到内核状态，开始处理I/O请求，操作系统检查内存中是否已经缓存了所需数据，若未命中则必须去磁盘读取
- 文件系统层将用户看到的逻辑文件偏移量映射为存储设备上的物理位置，由CPU配置DMA控制器（配置参数包括源地址、目标地址和传输长度），完成配置后CPU继续处理其他进程
- 硬盘或SSD控制器根据指令找到数据，先读到自己的片上缓存中，并进行预读操作，在这之后DMA控制器会向CPU申请总线控制权，直接将数据搬运到Page Cache中去，而不经过CPU寄存器
- DMA控制器通过中断信号通知CPU，CPU响应并执行中断程序，为了防止用户程序修改内核数据，操作系统会将数据从内核空间的Page Cache复制到用户空间的应用程序缓存区



3.

缺页异常是指虚拟内存管理中的核心机制，当程序试图访问一个已经映射在虚拟地址空间中，但当前并未加载到物理内存中的页面时，CPU 的内存管理单元（MMU）会拦截该访问，并触发一个异常信号通知操作系统

根据触发原因和页面状态的不同，缺页异常主要分为以下两类：

- 主要缺页：页面不仅不在物理内存中，而且也不在系统的任何缓存里
- 次要缺页：页面不在当前进程的物理内存中，但它已经在物理内存的其他地方

当CPU触发缺页异常时，操作系统会接管处理，根据异常类型采取不同的行动：

- 主要缺页：
  - 分配物理页框：操作系统在物理内存中寻找一个空闲的页框
  - 磁盘I/O读取：操作系统根据页表项中记录的外存地址，启动磁盘 I/O，将缺失的页面数据读入刚分配的空闲页框中

    - 更新页表：填入新的物理页框号，将该页表项置为 1，并刷新 TLB
    - 重新执行指令
- 次要缺页：在MMU或当前进程的页表中重新建立虚拟地址到已有物理页框的映射关系

4.

页面替换：当程序访问的页面不在物理内存中，且物理内存已满没有空闲页框时，操作系统必须选择一个已在内存中的页面将其换出，以腾出空间给新页面调入

页面替换的过程：

- 选替换页：操作系统运行页面替换算法，从当前内存中的页面里选择一个替换页面
- 检查并刷盘：检查该页面的修改位（Dirty Bit）。如果页面在内存中被修改过，必须先将其写回磁盘（刷盘）；如果未被修改，则直接释放
- 调入新的页：将所需的新页面从磁盘读入刚刚腾出的页帧中
- 更新页表：更新页表，将新页面的页表项标记为有效，并将牺牲页面的页表项标记为无效
- 重启指令：重新执行触发缺页中断的指令

找到合适的替换页面的方法：

- OPT（最佳置换算法）：一种理想化的理论算法，选择未来最长时间不会被访问的页面，或者以后永远不会被访问的页面进行淘汰，然而，这种算法在实际中是无法实现的，通常只作为衡量其他算法优劣的标准
- FIFO（先进先出算法）：总是选择驻留在内存中时间最长的页面进行淘汰，它认为较早调入内存的页面，不再被使用的可能性较大，但缺点是完全不考虑页面的实际使用频率
- LRU（最近最久未使用算法）：基于局部性原理，利用过去的页面访问历史来推测未来，它会淘汰最近一段时间里最久没有被访问过的页面，但实现成本较高，需要硬件支持来记录每个页面的访问时间或维护复杂的链表结构
- CLOCK（时钟算法）：操作系统将内存中的页面组织成一个循环队列（类似时钟表盘），并用一个指针循环扫描， 如果扫描到的页面“引用位”为1，说明最近被访问过，系统会将其引用位清零并给该页面“第二次机会”，然后继续扫描下一个；如果引用位为0，则直接选中该页面进行替换
- NRU（最近未使用算法）：为每个页面设置“引用位”和“修改位”，置换时，优先选择那些既没有被引用、也没有被修改过的页面进行淘汰
- Random（随机替换算法）：在无法确定哪些页面被访问概率较低时，直接随机选择一个页面换出（个人感觉太草率了。。）

思考题：在64位系统中，阶段二中的问题多数不会发生变化，差异主要集中于 虚拟地址空间 和 页表 相关的内容

- **虚拟内存**：
    - 虚拟地址空间大小：
        - 32位系统虚拟地址空间总大小为4GB（2^{32}字节），其中，用户空间（低3GB）内核空间（高1GB）
        - 64位系统虚拟地址空间理论上可以达到16EB（2^{64}字节），但实际硬件和操作系统并未完全利用，通常使用 48位 有效虚拟地址，提供高达 256TB 的寻址空间
    - 底层管理策略：
        - 32位系统的内核需要为超过1GB的内存分配页面时，就必须通过复杂的临时映射机制
        - 64位系统的内核可以直接映射 64GB 甚至 128GB 的物理内存，而不需要额外的映射开销
    - 安全性机制：
        - 32位系统多采用段式内存管理，地址空间狭小导致进程间互相干扰的风险更高

        - 64位系统采用纯页式虚拟内存，为每个进程分配了独立且庞大的逻辑地址空间，从硬件底层实现了进程间的完全隔离，受严格的权限控制，极大地提升了系统安全性
    - 数据结构占用空间：
        - 32位系统中一个 int 指针占 4 字节
        - 64位系统中一个 int 指针占 8 字节
- 页表管理
    - 页表层级结构：
        - 32位系统通常采用二级页表结构：虚拟地址被拆分为三部分：10位页目录索引 + 10位页表索引 + 12位页内偏移
        - 64位系统普遍采用四级页表结构，虚拟地址被拆分为五部分：9位全局页目录（PGD）索引 + 9位上层页目录（PUD）索引 + 9位中间页目录（PMD）索引 + 9位页表（PTE）索引 + 12位页内偏移
    - 页表项大小：
        - 32位系统：每个页表项通常占 4 字节
        - 64位系统：每个页表项通常占 8 字节

阶段三：致命一击 —— 深入调用栈与控制流挟持

【任务 3.1：解剖函数调用栈】

1.

栈在内存地址的增长方向：从内存的高地址向低地址方向增长

2.

栈帧：栈帧是栈操作的核心数据结构

栈帧的核心作用：栈帧的存在是函数能够被调用的基础，其主要作用体现在以下几个方面：

- 传递函数参数，使得被调用的函数能够获取到所需的数据
- 存储局部变量，保证不同函数调用之间的数据互相独立，互不干扰
- 记录返回地址，函数结束时通过读取该地址实现正确返回
- 保存与恢复执行上下文，在函数调用前后，使某些寄存器的值保持不变，从而保证主调函数的执行环境不被破坏
- 支持递归与嵌套调用，由于每次函数调用都会生成一个全新的、独立的栈帧，因此同一个函数可以多次递归调用自身而不会发生数据混乱，当递归结束时，栈帧会遵循“先进后出”的原则逐层释放

3.

ESP/RSP（栈顶指针）：始终指向 当前栈帧 的顶部（即内存中的低地址端），当发生函数调用时，系统需要为被调函数分配新的栈帧空间，此时 ESP/RSP 会向低地址方向移动，从而申请一块新的内存空间供新函数使用；当函数返回时，ESP/RSP 又会向高地址方向回退，释放这块空间

EBP/RBP（基址指针）：EBP 指向 当前栈帧中保存的旧 EBP 所在的位置（即栈帧的起始锚点）。它之上的位置依次存放着返回地址和调用者的参数；它之下是当前函数的局部变量。EBP 在函数执行期间保持不变，作为访问局部变量和参数的稳定参考点

4.

在函数调用的那一瞬间，程序通过 call 指令将 call 指令的下一条指令的内存地址（即函数执行完成后应该回去继续执行的位置）压入栈中，然后通过 call 指令修改指令指针寄存器的值，让程序跳转到被执行函数的起始地址去执行

返回地址在栈帧的EBP指针上方，具体地址根据操作系统的不同有所差异：在32位系统中为EBP+4，在64位系统中为EBP+8

【任务 3.2：赛博越权】

1.

gets(password)没有检查用户输入的长度，如果用户输入的字符串长度超过了password数组容量、gets() 会毫无警告地将多出来的字符继续写入内存中，继而会覆盖掉栈上相邻内存区域中的其他重要数据（在这道题中就是曾经的 RBP 和返回地址 Return Address），轻则导致程序崩溃，重则引发严重的安全问题

这种漏洞在网络安全中统称为：**栈溢出漏洞**

2.

为了让函数 verify_identity 结束后不返回main，而是直接跳转到 grant_access，字符串结构应该为：不少于16个字节的随意字符 | 曾经的 RBP (8字节) | 0x401156

这样一来，无论verify_identity内部如何执行，最后都会根据 0x401156 ，返回到grant_access，开启shell权限

【任务 3.3：最后防线 —— 现代内存保护机制】

1.

ALSR（地址空间布局随机化）：简单的解释就是：程序在每次启动时会随机化关键内存区域内的基地址，比如可执行代码，全局变量，栈，堆，共享库，这样就使得攻击者无法预测目标地址，从而基于固定地址的攻击失效

特点：

- 全组件随机化，包括程序代码，全局变量，栈，堆，共享库
- 高熵，Windows 64位系统上提供了38位有效随机熵（n位熵表示有 2^n 位随机结果）
- 默认强制开启，所有用户态进程全部启用
- 内核级集成，很难绕过

这种保护机制的思路很容易理解，就是让攻击找不到目标就完事儿了

后记

这一题的后记和以往的不太一样，是我边做前面的题目边补充的内容，因为在查找相关内容的时候总是会出现一些我不清楚的名词，所以我都一 一查清楚，列举在此处，最后进行了粗略的整理：

【CPU】

核心：CPU内部最重要的组成部分

- 功能：CPU所有的计算、接收/存储命令、处理数据等核心任务，都是由核心来执行的

- 多核架构：现在的CPU通常包含多个核心（如4核、8核），它们可以物理隔离、并行工作，从而大幅提升电脑处理多任务的能力

基础频率：CPU在正常、持续满负载下所有核心都能稳定运行的最低保证频率（频率越高，意味着理论上CPU处理指令的速度越快）

睿频：是英特尔（Intel）等厂商采用的一项动态加速技术，它允许CPU在处理高负载任务时，自动、动态地超过其基础频率运行

【主板】

芯片组：它是一组共同工作的集成电路，是主板的灵魂

- 功能：它负责将CPU与内存、显卡、硬盘以及其他外设连接起来，决定了主板级别的重要功能和整体性能
- 传统架构：以往芯片组主要由北桥（负责连接CPU、内存、显卡等高速设备）和南桥（负责连接硬盘、USB等低速外设）组成。随着技术进步，北桥的功能现在多已被整合进CPU内部，芯片组逐渐演变为单芯片设计

总线：计算机各部件之间传输数据、地址和控制信号的公共通道，根据功能主要分为三类：

- 数据总线（DB）：负责在各个部件之间传输实际的数据信息
- 地址总线（AB）：负责传送CPU发出的地址信息，用来指明数据要发送到内存或外设的哪个具体位置
- 控制总线（CB）：负责传递控制信号、时序信号和状态信息，用来协调和指挥各部件的工作

BIOS：固化在主板 ROM 芯片里的一组底层程序，它是计算机硬件与操作系统之间的“转换器”或“桥梁”，核心功能是：

- 上电自检：在开机时检查 CPU、内存、显卡等核心部件是否正常工作，如果有问题就会报警或提示
- 硬件初始化：唤醒并初始化硬件设备，使之进入准备好工作的状态，并为它们分配必要的系统资源
- 引导操作系统：寻找操作系统的引导程序，找到后就把控制权交给操作系统，随后电脑就会进入桌面

此外，BIOS 还提供了一个设置界面，允许高级用户或管理员自定义硬件配置，例如调整启动顺序、设置开机密码、管理电源或进行 CPU 超频等

模组：指的是一种“可插拔、可自由组合”的线材设计（特点：整洁方便，高度定制，易于维护）

【存储】

存储设备：主要用于长期或临时保存程序与数据，根据在系统中的位置和功能，主要分为两大类：

- 内存（主存储器）：特点是读写速度极快，但断电后数据会丢失，它主要用于临时存放正在运行的程序和数据
- 外存（辅助存储器）：它的特点是断电后数据不会丢失，用于长期保存文件。常见的有：
  - 固态硬盘：采用闪存技术，读写速度快，是目前的主流
  - 机械硬盘：采用磁性盘片存储，容量大且成本低，适合存储海量数据
  - 其他：如光盘（CD/DVD）、U盘等

P/E周期：指的是 NAND 闪存中 单个存储 块能够承受的“完整擦除+写入”操作的次数上限

- 物理损耗：闪存每次写入和擦除数据时，都需要施加高电压，这会对存储单元内部的氧化层造成微小的物理损伤，随着擦写次数的增加，氧化层受损严重，存储数据的可靠性就会下降，最终变成无法使用的 “坏块”

- 寿命：不同类型的闪存颗粒，其 P/E 周期差异巨大，例如，SLC 颗粒可达约 10万次，而目前主流的 TLC 颗粒通常在 1000~3000次左右，QLC 颗粒则更低

磨损均衡：字面意思，就是使得每个存储的磨损程度相对均衡

- 原因：如果没有任何管理，操作系统通常会反复向某些固定的存储块写入数据（例如系统文件、浏览器缓存等），导致这几个被频繁写入的块很快就会达到 P/E 上限而报废，而硬盘上其他大量的存储块可能还很“年轻”，一旦关键块损坏，整个硬盘就会提前报废
- 磨损均衡算法主要有两种：
    - 动态磨损均衡：主控芯片只关注空闲区域，每当有新数据需要写入时，算法会优先挑选那些擦写次数最少、最“年轻”的空白块来写入。这确保了新写入的数据不会过度消耗某几个特定的块
    - 静态磨损均衡：主控芯片不仅管空闲区，还会主动干预长期不更新的“冷数据”（如系统固件、极少修改的照片）。如果主控发现某些块因为存放了冷数据而长期闲置，而其他块因为频繁写入已经磨损严重，它就会把冷数据搬运到磨损较多的块上，从而把“年轻”的块腾出来，继续用于接收新的写入任务

【显卡】

显卡：核心作用是承担输出显示图形的任务，将计算机系统需要的数字数据转换成图像信号，并驱动显示器正确地显示出我们看到的画面

主要由以下四个部分组成：

- GPU核心：显卡的大脑，所有的计算任务都在这里完成，其架构和核心数量直接决定了显卡的性能
- 显存：相当于 GPU 的“临时工作台”，负责快速存储 GPU 计算需要的画面数据、纹理材质等，显存的容量和带宽对高分辨率游戏和 AI 计算至关重要
- 供电电路：保障显卡稳定运行的基础
- 输出接口：负责把 GPU 渲染好的画面最终输出到显示器上

根据硬件形态，显卡主要分为两类：

- 集成显卡（核显）：将 GPU 核心直接集成到了 CPU 内部，它没有独立的显存，需要和 CPU 共享系统内存。优点是功耗低、发热小，适合日常办公、看视频和轻度游戏；缺点是性能相对较弱，无法升级
- 独立显卡（独显）：是一张独立的板卡，拥有自己的 GPU 核心、独立的显存和供电电路，通常插在主板的高速 PCIe 插槽上。它的性能非常强大，能满足大型 3A 游戏、专业视频剪辑、3D 建模以及 AI 大模型训练等高负载需求，但相应的功耗和发热也更大

显存位宽：显卡的一个核心硬件参数，它指的是：显存在一个时钟周期内所能传送数据的位数，单位是比特（bit）

PCIe 插槽：主板上用来扩展硬件功能的物理接口，它是目前电脑中最核心的高速数据总线

- 最主要的用途就是安装独立显卡，可以提供超大带宽
- 每一个 PCIe 插槽都有自己专属的通道，设备之间互不干扰，因此数据传输速度极快且非常稳定

【DRAM】

字线（Word Line）：字线是水平贯穿一整行存储单元的导线，连接着这一行中所有晶体管的栅极（相当于开关的控制端）

字线相当于整行存储单元的“总开关”。当系统需要访问某一行时，会将对应的字线电压拉高，从而同时“打开”这一行所有的晶体管，允许它们与各自的位线连通

位线（Bit Line）：位线是垂直贯穿一整列存储单元的导线。它连接着这一列中所有晶体管的源极或漏极（相当于开关的数据端），并一直延伸连接到阵列底部的感测放大器、

位线是数据进出的通道，在读取时，电容上的电荷通过位线传导给感测放大器；在写入时，外部数据也是通过位线灌入目标单元的电容器中

行缓冲区：一个高速锁存器，它的作用是：

- 避免重复激活：如果后续访问的是同一行的其他列，无需再次激活行，直接从缓冲区读取，大幅降低延迟
- 数据暂存：将读取到的数据暂存，等待控制器按 位宽 分批传输

【性能/其他】

跑分：是指通过特定的基准测试软件（如安兔兔、Geekbench、3DMark等）对设备的CPU、GPU、内存等核心硬件进行量化测试，最终得出一个直观的分数

吞吐量：是指在某个时刻，在没有数据丢失（丢包）的情况下，网络设备能够接收并成功转发的最大数据速率。它反映了网络设备的实际传输能力和性能极限，通常以比特每秒（bps）或字节每秒（Bps）来衡量（显卡吞吐量：反映了在真实任务中，显卡每秒能处理多少数据、生成多少画面或完成多少次AI推理）

纹波：开关电源输出的并不是绝对平滑的直流电，而是带有微小波动的电压，这种波动就是纹波

外设：指连接到计算机主机以外，用来增强电脑功能或让操作更友好的设备，除了CPU和主内存，几乎所有部件都可以算作外设，主要分为三类：

- 输入设备：用于向电脑发送指令和数据，例如：键盘、鼠标、扫描仪、麦克风、摄像头等
- 输出设备：用于将电脑处理的结果展示给用户，例如：显示器、打印机、音箱等
- 存储外设：用于外部数据存储，例如：U盘、移动硬盘等

此外，还有很多内容我并没有过分深究，比如像栅极，漏极，源极等等

嗯，这一题已经写得足够长了

是的，来日方长……